

Алгоритмы
Функторы
Лямбда-выражения
И ещё раз алгоритмы

Алгоритмы

Обобщенные алгоритмы STL позволяют выполнять такие действия, как поиск, сортировка, вставка, переупорядочивание, удаление и копирование элементов объекта контейнерного класса.

Алгоритмы реализованы как **функции, которые работают с итераторами**. Это означает, что каждый алгоритм необходимо реализовать только один раз, и, как правило, он будет автоматически работать для большинства контейнеров, которые предоставляют набор итераторов (включая пользовательские классы контейнеров).

Чтобы использовать любой из алгоритмов STL, надо подключить заголовочный файл `algorithm`.

В функции стандартной библиотеки часто требуется передать два итератора, которые ограничивают диапазон элементов. К ним предъявляются такие требования:

- это должны быть итераторы одного и того же контейнера;
- от первого итератора ко второму можно перейти, применяя оператор ++.

Предполагается, что второй итератор при этом ссылается *за* последний элемент диапазона. Его нельзя разыменовывать оператором *, даже если он ссылается на валидный элемент.

Можно рассматривать пару итераторов как математический полуинтервал, где левый конец включается в диапазон, а правый — нет.

Все контейнеры имеют функции `begin` и `end`. Они возвращают итераторы, указывающие на начальный и *за* последний элемент всей последовательности элементов контейнера

Базовые алгоритмы

- **find** (<диапазон>, <что искать>) возвращает итератор на найденный элемент
- **count** (<диапазон>,<что считать>) – возвращает количество найденных элементов
- **copy**(<диапазон откуда>, <куда>) – копирует из одного контейнера в другой
- **sort**(<диапазон>) сортирует контейнер в заданном диапазоне
- **remove** (<диапазон>, <значение>) – «удаляет» элементы, соответствующие значению - попозже

ForwardIt **std::find**(ForwardIt first, ForwardIt last, const T& value)

Ищет последовательным перебором первое вхождение элемента, равного заданному. Результатом является итератор, который указывает на найденный элемент. Если же ничего не найдено, то возвращается правый конец переданного полуинтервала. Для пользовательского типа требуется определить **operator ==**. Находится в <algorithm>.

```
//Пример работы функции find. Найти последнюю позицию заданного элемента.  
//Поиск первого вхождения значения «2» с конца списка.  
//std::find возвращает итератор, указывающий на найденный элемент.  
//std::distance высчитывает количество переходов, которые необходимо пройти,  
//чтобы преодолеть расстояние между двумя итераторами.  
//Итераторы, переданные в distance, должны быть одного типа.
```

```
int main() {  
    std::list<int> v = {2, 7, 1, 8, 2, 8, 2, 4, 2, 7, 0};  
    std::cout << std::distance(std::find(v.rbegin(), v.rend(), 2), v.rend());  
}
```

`std::count` (ForwardIt first, ForwardIt last, const T& value)

Эта функция подсчитывает, сколько элементов последовательности равны заданному. Для пользовательского типа следует реализовать operator ==.

```
int main() {  
    std::list<int> v = {2, 7, 1, 8, 2, 8};  
    std::cout << std::count(v.begin(), v.end(), 8) << '\n';  
}
```

ForwardIt2 `std::copy`(ForwardIt1 first, ForwardIt1 last,
ForwardIt2 d_first);

Копирует содержимое одной последовательности в другую последовательность. Принимает три аргумента: обычную пару итераторов, задающих входной диапазон, и итератор, обозначающий начало выходного диапазона. Находится в <algorithm>.

```
int main() {
    std::vector<int> from_vector(10);
    //itoa заполняет диапазон [first, last) последовательно увеличивающимися
    //значениями, начиная с переданного (находится в numeric)
    std::iota(from_vector.begin(), from_vector.end(), 9);
    std::vector<int> to_vector;

    std::copy(from_vector.begin(), from_vector.end(), std::back_inserter(to_vector));
    std::cout << "to_vector contains: ";

    std::copy(to_vector.begin(), to_vector.end(),
              std::ostream_iterator<int>(std::cout, " "));
}
```

Как копировать map

В виду особой организации ассоциативного контейнера, скопировать один контейнер в другой можно с помощью output-итератора для вставки:

```
int main() {
    std::map<std::string, int> m1 = {"яблоки", 100}, {"груши", 13}, {"арбузы", 20},
                                   {"картофель", 15}, {"алыча", 22}};
    std::map<std::string, int> m3;

    std::copy(m1.begin(), m1.end(), std::inserter(m3, m3.begin())) );
    ы
    for (auto& a: m3) {
        std::cout << a.first << ' ' << a.second << ' ';
    }

    return 0;
}
```


std::sort (RandomIt first, RandomIt last)

Стандартом не оговорено, какой тип сортировки должен быть применён. Чаще используется алгоритм IntroSort. Это гибридный алгоритм, в котором используется три алгоритма для сокращения времени выполнения: Quicksort, Heapsort и Insertion Sort.

Алгоритм использует [быструю сортировку](#) и переключается на [пирамидальную сортировку](#), когда глубина рекурсии превысит некоторый заранее установленный уровень (например, логарифм от числа сортируемых элементов). Этот подход сочетает в себе достоинства обоих методов с худшим случаем $O(n \log n)$ и быстродействием, сравнимым с быстрой сортировкой. Для последовательностей, содержащих менее чем 16 элементов, применяется алгоритм [сортировки вставками](#).

Для сортировки пользовательского типа необходимо определить в нём оператор <.

В std::list реализована своя функция сортировки.

`std::stable_sort`

Стандартная сортировка `std::sort` не является стабильной, то есть при наличии равных элементов они всё равно могут меняться местами. При сортировке ресурсоёмких объектов пользовательских типов данный аспект может существенно повлиять на скорость работы.

В стандарте отмечено, что `std::stable_sort` является стабильной сортировкой, т.е. не меняет местами эквивалентные элементы при сортировке. Сложность ее в худшем случае $n * (\log(n))^2$ сравнений.

Алгоритмы с предикатом

- **find_if** (<диапазон>, <предикат>) возвращает итератор на найденный элемент, соответствующий предикату
- **count_if** (<диапазон>, <условие подсчёта>) – возвращает количество найденных элементов по условию
- **copy_if** (<диапазон откуда>, <куда>, <условие>) – копирует из одного контейнера в другой
- **remove_if** (<диапазон>, <предикат>) – «удаляет» элементы, соответствующие условию - попозже

Как написать предикат условия?

- Функция
- Функтор
- Лямбда

Пример работы с функцией в качестве предиката

Для поиска элементов, удовлетворяющих условию, требуется в качестве предиката передавать наименование функции (то есть указатель на функцию), которая принимает один параметр и возвращает bool:

```
bool isEven(int x) { return x % 2 == 0; }

int main () {
    std::vector v = {1,6,3,7,9,10};
    auto it = std::find_if(v.begin(), v.end(), isEven);
    std::cout << *it;
}
```

Потому что алгоритм find_if работает приблизительно так:

```
template<class Iterator, class Predicate>
Iterator find_if(Iterator first, Iterator last, Predicate pred) {
    while (first != last) {
        if (pred(*first))
            return first;
        ++first;
    }
    return last;
}
```

Пример работы сортировки с пользовательской функцией

```
bool descending(int a, int b) {  
    return a > b;  
}  
  
int main() {  
    std::vector<int> v = {5, 2, 8, 1, 9, 3, 7, 4, 6};  
  
    // Сортируем с использованием функции  
    std::sort(v.begin(), v.end(), descending);  
  
    std::cout << "После сортировки по убыванию: ";  
    for (int x : v) std::cout << x << " ";  
  
    return 0;  
}
```

Компаратор отвечает на вопрос: "Должен ли элемент 'a' идти перед элементом 'b'?" Если `comp(a, b)` возвращает `true`, значит, 'a' должен быть перед 'b'

Пример использования функции в качестве компаратора в контейнерах

```
bool descending(int a, int b) {  
    return a > b;  
}  
  
int main() {  
    // set с пользовательской функцией  
    std::set<int, bool(*)(int, int)> mySet(descending);  
    // priority_queue с пользовательской функцией  
    std::priority_queue<int, std::vector<int>, bool(*)(int, int)> pq(descending);  
}
```

При определении шаблонных параметров необходимо указывать ТИП. Поэтому следует писать указатель на функцию (это такой же тип как int или double)

Можно использовать using

```
using Comparator = bool(*)(const std::string&, const std::string&);
```

```
bool byLength(const std::string& a, const std::string& b) {  
    return a.length() < b.length();  
}
```

```
int main() {  
    std::map<std::string, int, Comparator> words(byLength);  
    words["One"] = 3;  
    words["Two"] = 3;  
    words["Four"] = 4;  
    for (auto& [a,b]: words) //!!! One 3 Four 4  
        std::cout << a << ' ' << b << '\n';  
}
```


count_if. Функция

```
bool isNegative(int x) {  
    return x < 0;  
}
```

```
int main() {  
    std::vector<int> v = {-5, 10, -3, 20, -1, 0, 30, -7};  
    auto count = std::count_if(v.begin(), v.end(), isNegative);  
    std::cout << "Отрицательных чисел: " << count << std::endl;  
    return 0;  
}
```

Функциональный объект

Объект функции (function object) или **функтор** (functor) представляет собой объект, который может вызываться как функция. В классе такого объекта должен быть определён оператор ().

```
class Print{
public:
    void operator()(const std::string& message) const {
        std::cout << message << std::endl;
    }
};

int main() {
    Print print; // определяем объект класса-функции
    print("Hello"); // выполняем как функцию
    print("World");
}
```

Преимущества

```
struct Counter {  
    int count = 0;  
    void operator()() {  
        ++count;  
        std::cout << "Вызов #" << count << '\n';  
    }  
};  
int main () {  
    Counter c;  
    c(); // Вызов #1  
    c(); // Вызов #2  
    c(); // Вызов #3  
}
```

Сохранение состояния

```
struct CompareByMod {  
    int mod;  
    CompareByMod(int m) : mod(m) {}  
    bool operator()(int a, int b) const {  
        return (a % mod) < (b % mod);  
    }  
};  
  
int main() {  
    std::vector<int> v = {15, 3, 22, 8, 17};  
    std::sort(v.begin(), v.end(), CompareByMod(10)); // сортировка по модулю 10  
    std::sort(v.begin(), v.end(), CompareByMod(5));  // по модулю 5  
}
```

Можно передавать параметры при создании

При объявлении переменной контейнера в качестве шаблонного параметра передаётся шаблонный класс, у которого реализован оператор ()

При работе с алгоритмами требуется передача именно объекта этого класса

Чтобы всё встало на свои места 😊

```
//Создание класса
struct testMy {
//Определение в классе оператора ()
    void operator()() {
        std::cout << "Hi\n";
    }
};

int main() {
//Создание временного объекта типа testMy
    testMy(); //очевидно, ничего выведено не будет
//Создание временного объекта типа testMy и вызов у него оператора ()
    testMy()(); //вывод Hi
}
```

Функтор vs функция в качестве шаблонного параметра:

```
template <typename T>
struct compare {
    bool operator() (T a, T b) { return a<b; }
};
```

```
template <typename T>
bool comp (T a, T b) {
    return a < b;
}
```

```
template<typename Key, typename Compare>
class set {
    Compare comp; // поле компаратора

public:
    // Для функтора: конструктор по умолчанию
    set() : comp(Compare()) {} // создаём объект функтора

    // Для указателя: нужен конструктор с параметром
    explicit set(const Compare& c) : comp(c) {} // сохраняем переданный указатель

    void insert(const Key& key) {
        // if (comp(key, someKey)) { }
    }
};

int main(){
    set<int, compare<int>> a;
    set<int, bool(*)(int, int)> b(comp);
}
```

В алгоритмы передаётся функциональный объект или указатель на функцию

```
struct testMy {
    bool operator()(int& a, int& b) {
        return a < b;
    }
};

bool testMy1 (int& a, int& b) {
    return a > b;
}

int main() {
    std::vector<int> v = {3, 6, 2, 7};
    std::sort(v.begin(), v.end(), testMy()); //функтор, то есть объект
    for(const auto& x: v) {
        std::cout << x << ' ';
    }
    std::sort(v.begin(), v.end(), testMy1); //функция
    for(const auto& x: v) {
        std::cout << x << ' ';
    }
}
```

Пример

```
//class_comp — это наименование типа
struct class_comp {
    bool operator()(const char& lhs, const char& rhs) const {
        return lhs < rhs;
    }
};

int main() {
    //Поэтому в контейнер передаём наименование типа class_comp. Если в этом типе
    //будет отсутствовать operator() или же оператор не будет соответствовать
    //семантике, будет ошибка компиляции
    std::map<char, int, class_comp> s;

    s['a'] = 20;
    s['b'] = 30;
    for(const auto& x: s) {
        std::cout << x.first << ' ' << x.second << ' ';
    }
    std::vector v = {5, 7, 3, -5, 0, -1};
    //сортировка принимает функциональный объект (или функцию), поэтому передаём
    //объект типа class_comp
    std::sort(v.begin(), v.end(), class_comp());
}
```


Использовать функтор можно как функцию

```
class Name {
private:
    char name[20];
    size_t sz;
public:
    Name(): name("\0"), sz(0) {}
    const char* operator()(const char* s){
        strcpy(name, s);
        return name;
    }
};

int main() {
    Name n; // определяем объект
    // результат работы оператора ( ) записали в переменную:
    const char* pch = n("Test");
    std::cout << pch << '\n';
    // результат работы оператора ( ) вывели на экран
    std::cout << n("New test") << '\n';
}
```

Стандартные функциональные объекты библиотеки <functional>

Стандартная библиотека содержит шаблоны классов для операций:

- Арифметических: plus, minus, multiplies, divides, modulus, negate.
- Сравнений: equal_to, not_equal_to, greater, less, ...
- Логических: logical_and, logical_or, logical_not.
- Побитовых: bit_and, bit_or, bit_xor, bit_not.

```
template <class T = void>
struct multiplies {
    constexpr T operator()(const T &lhs, const T &rhs) const {
        return lhs * rhs;
    }
};
```

Использование стандартных шаблонных классов-функциональных объектов

Такого плана классы используются в стандартных алгоритмах и контейнерах

```
//Опять же, в сортировку передаём функциональный объект класса greater
std::sort(
    names.begin(),
    names.end(),
    std::greater<std::string>{});

//в функцию accumulate передаём функциональный объект класса multiplies
auto const product = std::accumulate(
    nums.cbegin(),
    nums.cend(),
    1.0,
    std::multiplies<double>{});
```

Использование стандартных шаблонных классов в качестве компараторов (по умолчанию используется less)

```
template <typename Key, typename Value, typename Compare = std::less<Key>>
class map {
    . . .
};
```

```
int main() {

    std::map<char, int, std::greater<char>> f { {'c', 20}, {'f', 80}, {'e', 60}};
    f['a'] = 10;
    f['b'] = 30;
    for(const auto& x: f) {
        std::cout << x.first << ' ' << x.second << ' ';
    }

    std::set<char, std::greater<char>> s;

}
```

Лямбда-выражения

Лямбда-выражения представляют более краткий компактный синтаксис для определения объектов-функций. Формальный синтаксис лямбда-выражения:

[] (параметры) { действия }

Лямбда-выражение начинается с квадратных скобок. Затем, как в обычной функции, в круглых скобках идет перечисление параметров - типы и их имена. Также, начиная со стандарта C++14, для параметров можно указывать значения по умолчанию. За списком параметров, как и в обычной функции, в фигурных скобках помещаются действия, выполняемые лямбда-выражением.

То есть лямбда – это и есть функциональный объект.

```
int main() {  
    []() { std::cout << "Hello" << std::endl; }  
}
```

Каждый раз, когда компилятор встречается лямбда-выражение, он генерирует новый специальный тип класса, который представляет объект-функцию:

```
class __Lambda1234 {  
public:  
    auto operator()() const { std::cout << "Hello" << std::endl; }  
};
```

Таким образом, чтобы использовать лямбда-выражение, необходимо «вызывать» его через ():

```
int main() {  
    []() { std::cout << "Hello" << std::endl; }(); //Hello  
}
```

Имя лямбда-выражения

Чтобы лучше понять принцип определения и использования лямбда-выражения, определим имя выражению:

```
auto test = []() { std::cout << "Hello" << std::endl;};
```

В данном случае test – это функциональный объект специального типа:

```
class __Lambda1234 {  
public:  
    auto operator()() const { std::cout << "Hello" << std::endl; }  
};  
  
__Lambda1234 test = __Lambda1234{};
```

Поэтому чтобы использовать лямбда-выражение test, необходимо его вызвать, как и функцию:

```
test();
```

А если анонимное лямбда-выражение

При объявлении такого выражения:

```
[]() { std::cout << "Hello" << std::endl;};
```

Будет определён класс и сразу создан временный объект этого класса:

```
class __Lambda1234 {  
public:  
    auto operator()() const { std::cout << "Hello" << std::endl; }  
} __Lambda123452{};
```

При объявлении выражения с выводом:

```
[]() { std::cout << "Hello" << std::endl;}();
```

Будет определён класс, создан временный объект этого класса и сразу вызов оператора:

```
class __Lambda1234 {  
public:  
    auto operator()() const { std::cout << "Hello" << std::endl; }  
} __Lambda123452{};  
__Lambda123452.operator()();
```


Пример

```
int main() {
    std::vector v = {5, 7, 3, -5, 0, -1};
    std::sort(v.begin(), v.end(),
              [](int x, int y){return abs(x) < abs(y);});
    for (const auto& x: v) {
        std::cout << x << ' ';
    }
}
```

Как уже говорилось выше, лямбда-выражению можно определять имя, это будет переменная специального типа. Эту переменную можно передавать в функции, а также использовать в шаблонах.

```
int main() {
    std::vector v = {5, 7, 3, -5, 0, -1};
    auto cmp = [](int x, int y){return abs(x) < abs(y);};
    std::sort(v.begin(), v.end(), cmp);
    for (const auto& x: v) {
        std::cout << x << ' ';
    }
    //в map передаётся тип лямбда-выражения, а в конструктор сама лямбда
    std::map<char, int, decltype(cmp)> m(cmp);
}
```

С лямбда-выражением можно работать также, как и с обычной функцией

```
void do_operation(int a, int b, int (*op)(int, int)) {
    std::cout << op(a, b) << std::endl;
}

int main() {
    auto cmp = [](int x, int y){ return abs(x) < abs(y); };
    auto print = [](){std::cout << "Hello\n";};

    print();

    int x = 0;
    int y = 0;
    std::cin >> x >> y;
    if (cmp(x,y)) {
        std::cout << " x < y \n";
    }
    auto sum{[](int a, int b) { return a + b; }};
    auto subtract{[](int a, int b) { return a - b; }};

    do_operation(10, 4, sum); // 14
    do_operation(10, 4, subtract); // 6
}
```

Каждый раз, когда компилятор встречается лямбда-выражение, он генерирует новый внутренний специальный тип

```
int main() {  
    std::vector v = {5, 7, 3, -5, 0, -1};  
    auto cmp = [](int x, int y){return abs(x) < abs(y);};  
  
    //Поэтому в примере ниже decltype вернёт не тот тип, что у cmp  
    std::map<char, int, decltype([](int x, int y){return abs(x) < abs(y);})> m(cmp);  
    for (const auto& x: v) {  
        std::cout << x << ' ' << '\n';  
    }  
}
```

no matching constructor for initialization of 'std::map<char, int, decltype([](int x, int y) { return abs(x) < abs(y); })>'

В лямбда-выражения можно передавать параметры

Лямбда-выражение можно использовать сразу по месту определения

Если нет параметров, круглые скобки можно опустить

```
int main() {  
    auto increment {[](int& i) {  
        ++i;  
        return i;  
    }};  
  
    int x = 9;  
    std::cout << increment(x);  
  
    [](){std::cout << "Hello, ";}();  
    []{std::cout << "world\n";}();  
}
```

Пример использования лямбды по месту определения

```
int f(int& u) {  
    return u/5;  
}  
  
int main() {  
    int y = 0;  
    std::cin >> y;  
    const int x = [](int& u){  
        if (u < 0) return u*u;  
        else if (u >= 5) return f(u);  
        else return u;  
    }(y);  
  
    int arr[x];  
    for (int& i: arr) {  
        i = ++y;  
        std::cout << i << '\n';  
    }  
}
```

Возвращение значений из лямбда-выражений

По умолчанию компилятор сам определяет, значение какого именно типа будет возвращаться из лямбды:

```
int main() {  
    auto sum{[](int a, int b) { return a + b; }};  
  
    // вызываем лямбда-выражение  
    std::cout << sum(10, 23) << std::endl; // 33  
  
    // присваиваем его результат переменной  
    int result{sum(1, 4)};  
    std::cout << result << std::endl; // 5  
}
```

Можем явным образом указать возвращаемый тип:

```
auto sum {[](int a, int b) -> double  
    { return a + b; }};
```

Список захвата

Лямбды, как и обычные функции, могут получить доступ только к переданным в неё переменным и к глобальным идентификаторам. Чтобы лямбда могла обращаться к переменным из той области видимости, где она определена, используется список захвата. Переменные, указанные в списке захвата, не являются переданными в качестве параметров, но лямбда может их использовать. Список захвата помещается в `[]`.

Внутри квадратных скобок в определении лямбда-функции можно задать переменные из внешнего контекста, которые нужны для работы этой лямбда функции.

```
struct Point {  
    double x = 0;  
    double y = 0;  
    Point(double x, double y):x(x),y(y){}  
};  
int main() {  
    std::vector<Point> points{{2,5},{3,6},{2,1}};  
    double x = 0.0;  
    double y = 0.0;  
    std::cin >> x >> y;  
    auto cmp_sort = [x, y](const Point& a, const Point& b) {  
        return ((a.x - x)*(a.x - x) + (a.y - y)*(a.y - y)) <  
            ((b.x - x)*(b.x - x) + (b.y - y)*(b.y - y));  
    };  
    std::sort(points.begin(), points.end(), cmp_sort);  
    for (const auto& temp: points) {  
        std::cout << temp.x << ' ' << temp.y << '\n';  
    }  
}
```


Если лямбда захватывает переменные по значению, происходит их копирование. Так как `operator()` у лямбды помечен как `const`, изменять захваченные переменные, переданные по значению, нельзя.

Например, при определении такой лямбды:

```
auto add = [n](int x) { std::cout << x + n << std::endl; };
```

Сформируется приблизительно такой код

```
class ____Lambda1235 {  
public:  
    ____Lambda1235(int& arg1) : n(arg1) {}  
    auto operator()(int x) const {  
        std::cout << x + n << std::endl;  
    }  
private:  
    int n;  
};
```

Но по ссылке можно.

Потому что при определении такой лямбды:

```
auto add = [&n](int x) { n++; std::cout << x + n << std::endl; };
```

Сформируется такой код:

```
class ____Lambda1245 {  
public:  
    ____Lambda1245(int& arg1) : n(arg1) {}  
    auto operator()(int x) const {  
        n++;  
        std::cout << x + n << std::endl;  
    }  
private:  
    int& n;  
};
```

Можно «захватывать» все переменные из области видимости по значению:

```
int main() {  
    int n{10};  
    auto add = [=](int x) { std::cout << x + n << std::endl; };  
    add(4); // 14  
}
```

По ссылке:

```
int main() {  
    int n{10};  
    auto add = [&](int x) { std::cout << x + n << std::endl; };  
    add(4); // 14  
}
```

Можно комбинировать:

```
[&k, l, &m, n] // k и m – по ссылке, l и n – по значению  
[=, &m, &n](int x) // все по значению, а m и n – по ссылке  
[&, m, n] // все по по ссылке, а m и n – по значению}
```

mutable-параметры

Ключевое слово `mutable` может быть применено ко всей лямбда-функции, что сделает все её **переменные изменяемыми**

```
int main() {  
    int n{10};  
    auto increment = [=]() mutable {  
        n++; // увеличиваем значение внешней переменной n  
        std::cout << n << std::endl; // 11  
    };  
  
    increment();  
    std::cout << n << std::endl; // 10  
}
```

Захваты:

- Захват по ссылке;
- По значению;
- Захват с инициализацией (с C++14);
- Захват по умолчанию;
- Захват пакета `[obj...](){};`
- Захват текущего объекта `[this](){}.`

При работе с лямбда-выражением всегда надо обращать внимание на время жизни объекта, который она захватывает. Если обращаться к лямбде, которая работает с уже уничтоженным объектом, будет неопределённое поведение.

Замыкание (Closure)

Замыкание — это объект, который хранит в себе функцию и окружение (захваченные переменные). Другими словами, **замыкание** — это любая функция, которая **замыкается над** окружением, в котором она была определена. Это означает, что она может получать доступ к переменным, не входящим в список ее параметров

В C++ каждая лямбда, которая захватывает переменные, порождает замыкание.

```
auto makeCounter() {  
    int count = 0; // локальная переменная  
    // Замыкание захватывает count  
    return [count]() mutable {  
        return ++count; // меняем захваченную копию  
    };  
}  
  
int main() {  
    auto counter1 = makeCounter(); //создали функтор  
    auto counter2 = makeCounter(); //и ещё один  
  
    // Каждое замыкание "помнит" свой счётчик  
    std::cout << counter1() << std::endl; // 1  
    std::cout << counter1() << std::endl; // 2  
    std::cout << counter1() << std::endl; // 3  
  
    std::cout << counter2() << std::endl; // 1  
    std::cout << counter2() << std::endl; // 2  
}
```

Лямбда-выражение, использующее захват переменной, передать в функцию, принимающую указатель на функцию, не получится:

```
struct Point {
    double x = 0;
    double y = 0;
    Point(double x, double y):x(x),y(y){}
};

void do_operation(const Point& a, const Point& b, bool (*op)(const Point&, const Point&)) {
    std::cout << op(a, b) << std::endl;
}

int main() {
    std::vector<Point> points{{2,5},{3,6},{2,1}};
    double x = 0.0;
    double y = 0.0;
    std::cin >> x >> y;

    auto cmp_sort = [x, y](const Point& a, const Point& b) {
        return ((a.x - x)*(a.x - x) + (a.y - y)*(a.y - y)) <
               ((b.x - x)*(b.x - x) + (b.y - y)*(b.y - y));
    };

    do_operation(points[0], points[1], cmp_sort); //!!!
}
```

candidate function not viable: no known
conversion from '(lambda at main.cpp:540:21)' to
'bool (*)(const Point &, const Point &)'

Лямбда с захватом хранит состояние (захваченные переменные). Обычный указатель на функцию не может хранить состояние — он только указывает на код

Обобщённая лямбда

это лямбда-выражение, в котором как минимум для одного параметра в качестве типа указано слово `auto` или выражения `auto&` или `const auto&`:

```
auto add = [](auto a, auto b) { return a + b; };
auto print = [](const auto& value) {std::cout << value << std::endl; };

std::cout << add(2, 3) << std::endl; // 5 – складываем числа int
std::cout << add(2.2, 3.4) << std::endl; // 5.6 – складываем числа double

print("Hello");
print(4);
```


std::function

При использовании объектов-функций и лямбда-выражений следует помнить, что они не эквивалентны указателям на функции. Чтобы упростить совместное использование указателей на функции, объектов-функций и лямбда-выражений модуль <functional> определяет шаблон **std::function<>**

Шаблонный класс `std::function` — это универсальная полиморфная обертка над функциями. Экземпляры `std::function` могут хранить, копировать и вызывать любые функциональные объекты — функции (через указатели на них), лямбда-выражения, функторы, а также указатели на функции-члены и указатели на члены данных. Вызываемый объект называется целевым объектом `std::function`. Если `std::function` не содержит целевого объекта, она называется пустой.

Попытка вызова пустой `std::function` вызывает исключение `bad_functional_call`.

```
#include <functional>
int sum (int x, int y) {
    return x + y;
}
struct Multiply {
    int operator() (int x, int y) {
        return x*y;
    }
};

int main() {
    int x = 0;
    int y = 0;
    std::cin >> x >> y;
    std::function <int(int, int)> f;

    f = sum;
    std::cout << f(x, y) << '\n';

    f = Multiply();
    std::cout << f(x, y) << '\n';

    f = [](int x, int y) {
        return x - y;
    };
    std::cout << f(x, y) << '\n';
}
```

Ещё пример

```
void do_operation(const Point& a, const Point& b,
    std::function<bool(const Point&,const Point&)> op) {
    std::cout << op(a, b) << std::endl;
}

int main() {
    std::vector<Point> points{{2,5},{3,6},{2,1}};
    double x = 0.0;
    double y = 0.0;
    std::cin >> x >> y;

    auto cmp_sort = [x, y](const Point& a, const Point& b) {
        return ((a.x - x)*(a.x - x) + (a.y - y)*(a.y - y)) <
            ((b.x - x)*(b.x - x) + (b.y - y)*(b.y - y));
    };

    do_operation(points[0], points[1], cmp_sort);
}
```